

Why RAG Sometimes Fails

Introduction

RAG (Retrieval-Augmented Generation) systems improve Large Language Models by allowing them to retrieve external knowledge before generating answers.

Basic RAG Flow:

User Query



Vector Search



Retrieved Documents



LLM Generates Answer

Although RAG systems are powerful, they are not perfect.

Even when correct documents exist in the database, the system may still generate poor answers.

Why RAG Systems Fail

1. Retrieval Failure

What is Retrieval?

Retrieval is the process of searching and fetching relevant documents from a vector database.

Problem

Sometimes the retriever:

- misses important documents
- retrieves irrelevant content
- fails to understand query intent

As a result:

- the LLM receives poor context
 - generated answers become inaccurate
-

Real-Life Example

User Query:

"How to reset Kafka consumer offset?"

But retrieval returns:

- Kafka cluster setup
- Kafka broker architecture
- Kafka monitoring

instead of offset reset commands.

The final answer becomes weak or incorrect.

2. Chunking Failure

What is Chunking?

Chunking means splitting large documents into smaller pieces before embedding them.

Why Chunking is Important

LLMs cannot process extremely large documents efficiently.

So documents are divided into:

- paragraphs
 - sections
 - small chunks
-

Problem with Poor Chunking

If chunks are:

- too small → context gets lost
- too large → irrelevant information increases

then retrieval quality drops.

Example

Suppose:

- one chunk contains half command
- another chunk contains explanation

The retriever may only fetch one incomplete chunk.

This causes incomplete answers.

3. Embedding Problems

What are Embeddings?

Embeddings convert text into numerical vector representations.

These vectors help the system understand semantic meaning.

Problem

Sometimes embeddings:

- fail to capture true intent
 - misunderstand technical terminology
 - confuse similar concepts
-

Example

Query:

"Kafka offset reset"

Embedding model may incorrectly relate it to:

- monitoring
- broker management
- topic creation

instead of consumer offset operations.

4. Top-K Retrieval Issues

What is Top-K?

Top-K means retrieving the top matching documents from vector search.

Example:

- Top 3 documents
 - Top 5 documents
 - Top 10 documents
-

Problem

Even if correct documents exist:

- they may not appear in Top-K results
- lower-quality documents may rank higher

This reduces answer accuracy.

Example of Failure

User Query



Vector Search



Wrong Top-K Documents



LLM Generates Incorrect Answer

Solution: Reranking

What is Reranking?

Reranking is an additional filtering step used after vector search.

Instead of directly sending retrieved documents to the LLM:

- documents are re-evaluated
- relevance is recalculated
- best documents are selected

This improves answer quality.

Reranking Workflow

User Query



Vector Search



Top 10 Docs



Reranker



Best 3 Docs



LLM

How Reranker Works

Step-by-Step

The reranker compares:

User Query ↔ Retrieved Document

one document at a time.

It calculates:

- semantic similarity
- contextual relevance
- usefulness

and assigns relevance scores.

Example of Reranker Scoring

Document	Score
Kafka architecture	0.45
Broker monitoring	0.22
Offset reset command	0.98

Meaning of Relevance Score

Definition

A relevance score represents how useful and contextually related a retrieved document is to the user's query.

Higher score = better match.

Why Reranking is Important

Benefits

Better Context Selection

Removes low-quality retrieved documents.

Improved Accuracy

LLM receives more relevant information.

Reduced Hallucinations

Correct context reduces fake/generated misinformation.

Better Semantic Matching

Understands meaning better than simple vector similarity alone.

Real-Life Example

HRGPT Resume Search

User Query:

"Python developer with FastAPI and LangChain experience"

Vector search may retrieve:

- Python developer
- Backend engineer
- AI engineer

Reranker improves results by selecting resumes most relevant to:

- FastAPI
- LangChain
- AI backend development

Reranking vs Vector Search

Vector Search vs Reranking

Comparison of retrieval quality and contextual understanding in RAG systems.

Noise Reduction

Vector Search4

Reranking10

Vector Search

Reranking

036912Semantic UnderstandingContext AccuracyNoise ReductionPrecisionFinal Answer

Quality

Common Reranker Models

Cross-Encoder Models

These models compare:

- query
- document

together for deeper contextual understanding.

Popular Rerankers

- Cohere Rerank
 - BGE Reranker
 - Sentence Transformers Cross-Encoders
 - Jina AI Reranker
-

Real-Life Use Cases of Reranking

Enterprise Knowledge Chatbots

Improves internal document retrieval.

AI Search Engines

Returns more accurate search results.

Customer Support AI

Fetches correct troubleshooting steps.

HR Systems

Improves resume and policy document retrieval.

Conclusion

RAG systems can fail even when correct documents exist because:

- retrieval may fail
- chunking may break context
- embeddings may misunderstand meaning
- Top-K retrieval may select wrong documents

To solve this problem, reranking is used.

Rerankers:

- compare query and documents deeply
- assign relevance scores
- select the best contextual documents

This improves:

- retrieval quality
- answer accuracy
- semantic relevance
- overall RAG performance

Reranking is now a critical component in modern production-grade RAG systems.